

Background

- Competition Name: CAFA 6 Protein Function Prediction
- Team Name: protein_team
- Private Leaderboard Score: 0.48731
- Private Leaderboard Place: 2

Team:

Anton Vakhrushev, UAE, Dubai. Btbpanda@gmail.com

Background:

- Education: math methods in economics
- Experience: 10+ years of Data Science and RnD in banking and finance
- Kaggle experience: 10 years

One of my research areas is applying GBDTs for extreme multioutput tasks, that motivates me to enter the competition.

Sergei Fironov, Russia, Saint-Petersburg. ifserge@gmail.com

Background:

- Education: Bachelor of Physics
- Experience: 10+ years of RnD in Mass Spectrometry field, 7 years of DS application experience for industry, e-commerce and user experience improvement
- Active kaggle experience: 8 years

Initially, I was interested in the competition, believing that I could develop large models for the protein sequence. But the approach didn't work.

Alexander Chervov, France, Paris, Institute Curie. al.chervov@gmail.com

Background:

- Education: PhD in mathematics
- Experience: 10+ academia research in mathematics, 10 years of industrial data analysis and data science, 4 years academia research in bioinformatics and data science
- On kaggle experience: 5 years

I am interested in data science application to bioinformatics. I hope experience on that challenge can be applied to other tasks like prediction of kinase-sites phosphorylation by proteins, which is related to some my projects in Curie.

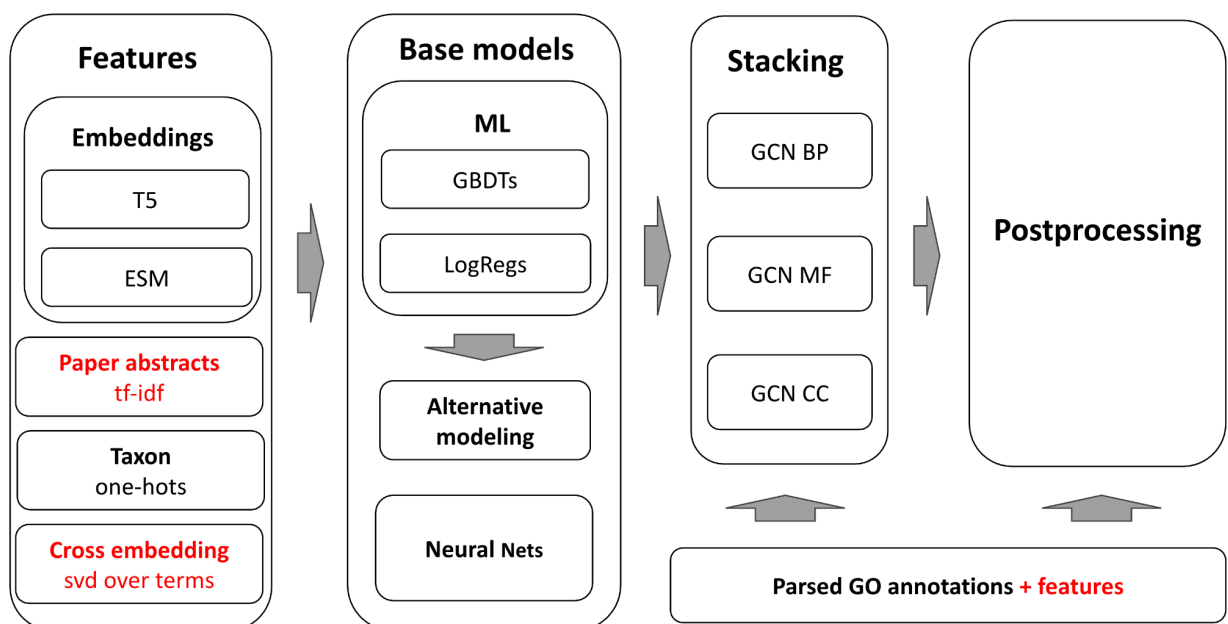
Solution overview and relationship with CAFA 5

The key parts of our solution is inspired by the algorithm we built during the CAFA 5 competition <https://github.com/btbpanda/CAFA5-protein-function-prediction-2nd-place>.

After the first look at the dataset we found that CAFA 5 and CAFA 6 are highly overlapped. Simplifying a bit, we can say that CAFA 6 is the up-to-date labelled subset of CAFA 5. So, the obvious idea to train is to try to rollback the sample definition to a large CAFA 5 like sample. It is not definitely a good idea from the performance point of view but something worth testing. For easy reference we would call:

- **Train:** train dataset provided at CAFA 6 competition
- **Old train:** the subset of CAFA 5 proteins that were not represented in Train. Up-to-date labelling was built from the actual uniprot database.

General approach and used methods are represented on the Figure below:



Novel parts introduced in CAFA 6 solution are marked as red. The other difference is in the number of models. We built “Train” and “Train + Old train” data variants of base models, different embedding combinations for the base models, and more seeds of the stacker model.

By the end of competition we can admit that the most important new update comes from the tf-idf articles part. The rest of the improvements are minor and mostly increase the solution robustness rather than bring new knowledge.

Below we describe all the steps in more detail.

Sequence embedding and base models features

Below we listed all the features used in base models only. Additional stacker features are provided in the corresponding section.

1. Protein DL embedding

We tried to use the following embedding list:

- T5
- esm2-small

We used pre-trained models only. To our surprise, fine-tuning was not helpful in this task

2. Articles TF-IDF embedding

Downloading abstracts: For each UniProt protein accession, PubMed identifiers (PMIDs) are retrieved from UniProt using the `lit_pubmed_id` field via the UniProt REST API. Duplicate PMIDs are removed, and the corresponding article titles and abstracts are then downloaded in batches from PubMed using the NCBI Entrez API.

Articles-derived protein embeddings: Using the downloaded article titles and abstracts, we computed TF-IDF features with 5,000 dimensions for each protein across all datasets. These TF-IDF vectors were then used as articles-derived protein embeddings in the downstream models.

The articles-derived TF-IDF embeddings were used cautiously, as they may introduce a minor data leakage risk. Specifically, we included only one PyBoost model in the final stack that used these TF-IDF embeddings concatenated with T5.

3. Taxon one-hot representation

In addition, we concatenated one-hot taxon features to the protein embedding. We selected only taxons that are good enough represented in both train and test features, around 30 totally, other taxons were merged into a single group. According to our estimation, adding one-hot taxon data improves the CAFA score of base models about 0.02-0.03 (based on CAFA5, didn't perform extra evaluation on actual data)

4. Cross ontology terms SVD

Another innovation introduced in CAFA 6 compared to the previous competition is evaluation over 3 independent test sets: no knowledge, limited knowledge, and partial knowledge. In 2 of those test sets we aim to make a prediction for the proteins that are not completely new but already have some labelling. The obvious idea was to utilize this knowledge somehow. We put some effort on making it work, but the effect was quite limited.

The following approach we find a little bit useful, at least consistently not making it worse: let's focus on boosting the model performance in a limited knowledge setup. In this scenario if we make a prediction for BP we already have some labels in MF or CC or both. We can do one-hot label representations of MF/CC targets, compress it via SVD, and use it as a features to predict BP. We repeated this trick for each single ontology using 512 SVD of two rest ontologies as features.

Validation

Our validation scheme could be separated on 2 parts:

Base models validation: we were not able to create a validation scheme better than a simple 5 fold CV. We had some experiments on the topic but other CV schemes lead to the models with less LB score.

Stacking models validation: we didn't use any local validation set to fit the stacker model. Instead, we used 100% data to train and rely on Public LB scores. Generally, it is a bad practice. However, we compensate it with a strong prior knowledge of model behavior obtained during CAFA 5, and with stochastic weighted average technique to make the learning procedure more robust.

Base models

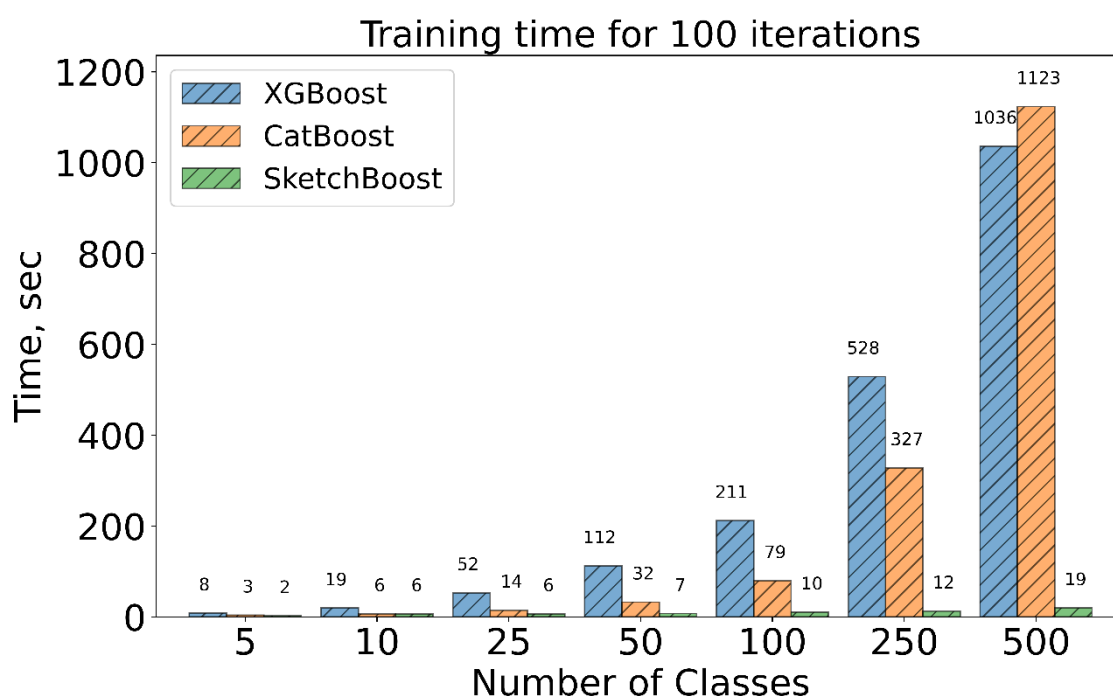
All models were trained to multi-label task, where target matrix of shape $n_{proteins} \times n_{terms}$ is predicted using feature matrix $n_{protein} \times n_{features}$ using binary cross entropy loss. In addition, some of models were trained using alternative conditional approach described in the next section

To train each model we selected top frequent terms from each ontology, top amount depends on the model type and provided below.

Py-Boost

The best performing models on both CV and LB are from the Gradient Boosting family. We used my own GBDT implementation called **py-boost**. I made it a few years ago especially to deal with

extreme multi-output datasets since it was my main research area at that time. It works on GPU only and it is able to train multi-label tens on even hundreds times faster than popular well known implementations. You can check **py-boost** on the [github](#) or read our NeurIPS [paper](#), where we explain all the strategies to speed-up multi-output training. The following Figure illustrates the speed up of our method (**SketchBoost** refers to the fast algorithm implemented inside the **py-boost** library).



Locally we were able to fit 4.5k output (3000/1000/500) **py-boost** on a single V100 32GB GPU and it takes about 2 hours for a single fold.

Logistic Regression

We also trained a simple 13.5k output Logistic Regression (10000/2000/1500). It shows much less performance than GBDT on the popular terms, but is able to perform on rare outputs.

Neural Networks

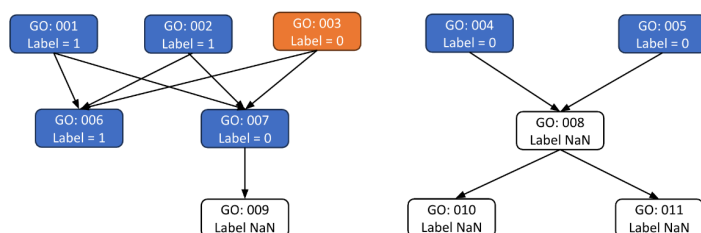
For the neural network models, we kept the architecture and training code the same as in the public notebook [Pytorch,Keras,Etc 3 Blend, CAFA metric, etc.](#) The main difference was the best cross-validated combination of hyper parameters averaged many times. However, instead of selecting only the top 2,000 terms, we used the same 13.5K targets as in the linear regression models. T5 and ESM embeddings were used as input features, without one-hot taxon features.

Alternative modelling approach: predicting the conditional probabilities

This approach was discovered at the beginning of the CAFA 5 competition. That's why it makes some wrong assumptions about the data. But somehow it becomes useful for both CV and LB. The main advantage of this approach is utilising the OBO graph on the inference phase and it helps to make a prediction even for terms that were not used in training. Here are the main points:

1. We assume that term can exist for the protein only **if at least one of its parents exists**. This is wrong. In practice, if the term exists, all its parents exist too because of the propagation rules.
2. We reformulate a classic multi-label scheme where the target matrix of shape (n_protein, n_terms) consists of 0 and 1 to the new scheme. Now targets can be 0, 1 and NaN. The term for the protein will have NaN value in the matrix if there is no parent term with value 1.
3. During model training phase NaN cells in the target matrix are masked and ignored
4. Now, our model outputs **the conditional probabilities of the term in case at least one of its parents exists**. On the inference phase we need to transform it back to the raw probabilities
5. Transformation is made in the order defined by the graph. When we process the term, all its parents are already processed and have raw probabilities. All terms are included in the scheme, even if they are not used in training. For the terms that were not used for training, we used prior mean.
6. While processing the term, we make another wrong assumption, that parents' probabilities for the term are independent. But if we assume that, according to [1], we can calculate raw probability for term as $P(GO: N) = P_{cond}(GO: N) * \left(1 - \prod_{K \in Par(N)} 1 - P(GO: K)\right)$ Remember that while we process the term, all its parents already have raw probabilities calculated.

The following Figure represents the main difference between classic multi label and alternative conditional scheme:



Key assumptions:

- 1) Label could be 1 or 0 only if one of the parents is 1. Otherwise label is set to NaN
- 2) Events $(GO: k = 1)$ for all K are independent

GO: 003 could not be 0 in real because GO: 006 is 1. Assumptions are wrong but the approach still works

Model type 1: Classic Multilabel, $P(GO: N = 1)$

	GO: 001	GO: 002	GO: 003	GO: 004	GO: 005	GO: 006	GO: 007	GO: 008	GO: 009	GO: 010	GO: 011
ProtID	1	1	0	0	0	1	0	0	0	0	0

Model type 2: Conditional, $P_{cond}(GO: N = 1 | (GO: Par0 = 1) \text{ or } \dots \text{ or } (GO: ParK = 1))$

	GO: 001	GO: 002	GO: 003	GO: 004	GO: 005	GO: 006	GO: 007	GO: 008	GO: 009	GO: 010	GO: 011
ProtID	1	1	0	0	0	1	0	NaN	NaN	NaN	NaN

Relationship between Type 1 and Type 2

$$P(GO: N) = P_{cond}(GO: N) * \left(1 - \prod_{K \in Par(N)} 1 - P(GO: K)\right)$$

On the Figure term GO:003 marked red, because it represents the situation that could not be observed in the data – GO:003 could not be 0 because its child GO:006 is 1. The figure represents our view and motivation on the moment of discovering the approach, not the real situation.

That approach shows well performing on CAFA 5, so, it was accepted without any additional evaluation.

Extensive models list

GBDT over DL embeddings: totally, we have 12 models in this category. The models were trained over:

- Train Data / Train + Old Train data
- T5 + taxon / ESM + taxon / T5 + ESM + taxon
- Raw / Conditional multilabel

Models were trained to predict 4.5k outputs

Finally, to reduce the number of inputs to stacker we averaged models over train dataset and embedding types and keep only 2 predictions:

- GBDT avg conditional
- GBDT avg raw

Log reg over DL embeddings: we have the the same 12 log regs as GBDT, the difference is it was trained to predict 13.5k outputs, so finally after averaging we have 2 predictors:

- Log reg avg conditional
- Log reg avg raw

Feed forward NN over DL embeddings: there are 2 models trained on CAFA 6 data only and combined T5+ESM embeddings:

- NN conditional
- NN raw

Cross SVD embeddings: we have 12 models in this category. Models are trained over:

- GBDT / Log reg
- Raw / conditional
- BP / MF / CC

Each ontology predictor uses 4 of them - trained on 2 rest ontologies SVD, so finally we have:

- Log reg cross SVD conditional
- Log reg cross SVD raw
- GBDT cross SVD conditional
- GBDT cross SVD raw

GBDT over DL + articles: we have a single GBDT model that uses articles tf-idf embedding + T5 + taxons. Model is trained on CAFA 6 Train only

Summary: 11 predictors for each ontology stacker

Stacking with GCN

We used a graph convolution network to aggregate all the predictions. It is trained for the node classification task where each node is a term and each protein is a graph (but all the proteins share the same adjacency matrix). As the node features we used base models predictions together with node embedding trained from scratch. We also added GO annotations features described in the next section.

Features are:

Base model predictions: 11 base model predictions. Each model prediction is transformed to 4 channels. The important assumption here: **we assume all base models have predictions for all terms**. If the model doesn't have it is filled with prior values. Priors are computed over Train or Train + Old train regarding the train sample used to fit the model. Channels are:

- Prior prediction flag
- Prediction logit value
- For conditional: propagation type 1 logit, for raw just copy prediction logit
- For conditional: propagation type 2 logit, for raw just copy prediction logit

More details are provided on the Figure below

GO annotation features: 38 binary features are computed based on GOA electronic labelling. Details are described in the next section. Simply saying, each binary feature represents what kind of relationship between protein and term is available in GOA

Learnable term embedding: just learnable from scratch embedding of term of size 8

Summary: each protein / term pair is represented by $11 * 4 + 38 + 8 = 90$ channels

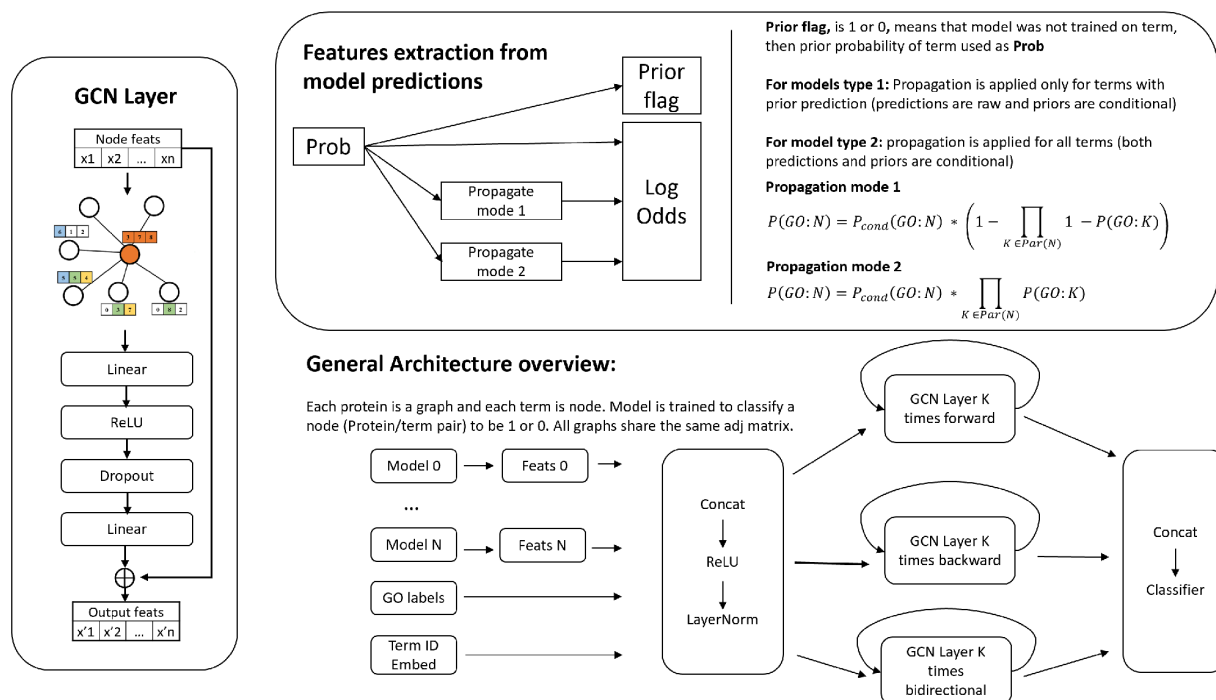
Targets are:

Binary labels of protein/term pair. All terms are used in training. Models were trained using binary cross entropy loss.

Train sample

Stackers were trained independently for each ontology using only CAFA 6 train data. Multistart with 2 random seeds were used, the prediction results are averaged in the end. Another important thing we should mention is about the metric. As far as we understand, we are evaluated only on protein/ontology pairs that are experimentally found. So, if we predict a term from ontology that does not exist, we will not get any penalty at all! That means, that we need actually to estimate **the conditional probability of a term in case when its ontology exists**. So, the correct way to fit a final stacker model for ontology X is to truncate the sample and take only the proteins that contain the terms from ontology X. It gives some small boost to the score and speeds-up the computations.

The following Figure represents the details of stacking model implementation:



GO annotations

We discovered the GO annotations dataset provided by the [link](#). They provide not only labelling but also the evidence codes of each term. We separated the codes that Kaggle suppose to be

experimental¹ from the electronic codes. So, we can use electronic labelling as the features to predict experimental labels given by Kaggle. From our analysis we discovered that about 30% of electronic labels become experimental, so using it as a model feature performs better than just adding it as is.

The electronic evidence codes provided by the labelling are not equal. They differ by:

- The evidence code like IEA / IBA / ISO / ISS etc. Even if we don't understand the difference, may be ML models can make it work
- Type of relation like part_of / acts_upstream / is_active_in
- Evidence counts. A protein could have multiple evidence for a term with different codes. Having multiple evidences highly increase the probability of transformation to experimental, for example for 3 codes it is more than 50% compared to 30% for 1 code

Using those information we computed 38 binary features like:

- Protein has at least 1/2/3/4/5 evidences for term (propagated labels counts)
- Protein has at least 1/2/3/4/5 evidences for term (within only raw labels)
- Protein is related with term as "part_of"
- Protein has an evidence of type IEA for term
- ...

This is only the idea of feature generation provided. For the extensive list of features please see our code.

GOA annotations releases: we started our work when release 226 was available. According to our knowledge, this release was also used to build the original competition data. Finally, by the end of the competition the latest available version was 228. So, this version was used to make a final prediction for the test set

One thing we need to mention here. Since the 228 data was released after competition started, some very small amounts of test proteins get experimental labels in the latest version of GOA. If we detect such cases, we just add it to the final submission file without impact on the training procedure.

¹ The exact experimental codes are: IDA IMP TAS IPI IEP IGI IC EXP HTP HDA HMP HGI HEP. With this list we were able to exactly reproduce the train labels provided by the hosts

Postprocessing

We also used the OBO graph to make a post processing. The problem of ML model prediction of protein terms is that it is inconsistent. Following the propagation rule that is applied to the target if the term exists, all its parents are assumed to exist too. So, a consistent model will never predict the probability for parents lower than term probability. But our models don't care about it at all. So we can manually fix this situation. Our final term prediction is **the average of term probability, maximum propagated children probability, and minimum propagated parents probability.**

The figure below illustrated the postprocessing details:

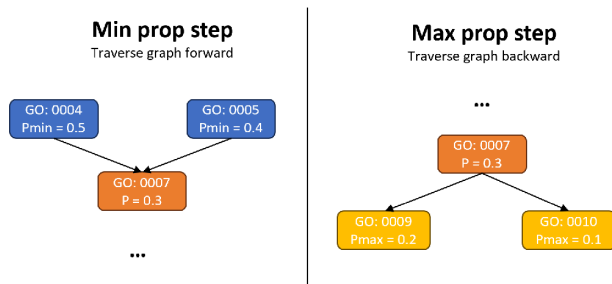
General rules:

Postprocessing is made in order defined by OBO Graph. After term N is processed, its probability is modified inplace and for next terms $P_{min}(N)$ will be used instead of $P(N)$. Min and max propagations are done separately and then results are averaged

$$P_{min}(N) = \text{Min}\{P_{min}(k), k \in \text{Par}(N)\} * 0.7 + P(N) * 0.3$$

$$P_{max}(N) = \text{Max}\{P_{max}(m), m \in \text{Chld}(N)\} * 0.7 + P(N) * 0.3$$

$$P_{post}(N) = \frac{P_{min}(N) + P_{max}(N)}{2}$$



Example:

For term GO:0007 we will calculate the final value as:

$$P_{min}(N) = \text{Min}\{0.5, 0.4\} * 0.7 + 0.3 * 0.3 = 0.37$$

$$P_{max}(N) = \text{Max}\{0.2, 0.1\} * 0.7 + 0.3 * 0.3 = 0.223$$

$$P_{post}(N) = (0.37 + 0.223) * 0.5 = 0.296$$

Simplified model

We believe, it does not make sense to choose one best and simple approach, since:

1. There is no need to make fast and efficient online inference for such kind of research task
2. Any single model scores much less that combination of multiple approaches

We consider using all the models and data sources to evaluate all the potential protein functions. Using GO electronic labelling as a feature in the stacker model is valid knowledge to predict, which terms will be experimentally found.

Source Code

Solution code is open source now and available at <https://github.com/btbpanda/CAFA6-updates>

Appendix: additional study obtained from CAFA 5

As we strongly rely on the CAFA 5 results we didn't perform extensive study of each component in terms of training time and impact of the models. To get an approximate understanding of this we refer to our CAFA 5 solution. This can show an order of effect and complexity for the main solution components

Ablation (based on public LB)

Here is brief evolution of our solution (scores are approximate):

1. Single py-boost over T5 + alternative modelling: 0.51
2. Step 1 + taxon data: 0.53
3. Blend of 2 py-boost and 2 LogRegs (classic and conditional): 0.55
4. Step 3 + GOA test leakage: 0.57
5. Step 4 + NN base model: 0.59
6. Stacking + GOA test leakage: 0.62

Model execution time

Here we provide the approximate runtime of each step. All time is provided for single V100 32GB GPU:

1. **Preparation step.** All the preprocessing takes about **1-2 hours**, excluding the time for downloading FTP GOA data sources (it takes maybe 10 hours)
2. **Embedding** inference takes about **3 hours for each model** type. If 2 GPUs are available, inference could be done in parallel
3. **Base models** (all time listed for 5 fold CV). All models could be trained in parallel. Inference is quite fast and takes much less time than train (few minutes for each model):
 - a. **py-boost:** 4 models, each takes **15 hours**
 - b. **LogReg:** one model **10 hours**, another **2 hours**
 - c. **Neural Net:** one model blended 12 times **1 hour**
4. **Stacking train** (single model for each ontology), could be trained in parallel:
 - a. **BP: 13 hours**
 - b. **MF: 5 hours**
 - c. **CC: 2 hours**
5. **Stacking inference** (4 times TTA), could be inferred in parallel:
 - a. **BP: 1.5 hours**

b. MF: 40 mins

c. CC: 20 mins

6. Postprocessing: 5 mins